

Day 31–33: Collections Deep Dive

Learn:

- Set
- HashSet
- LinkedHashSet
- TreeSet
- Map
- HashMap
- LinkedHashMap
- TreeMap

Practice:

- Frequency counter
- Duplicate removal

1. What problem does Collections Framework solve?

Suppose you want to store 5 student names.

Without Collections:

```
String student1 = "Ravi";
```

```
String student2 = "Ram";
```

```
String student3 = "Sita";
```

```
String student4 = "John";
```

```
String student5 = "Priya";
```

If there are 10,000 students, this approach becomes difficult.

So we use an **array**:

```
String[] students = {
```

```
    "Ravi",
```

```
    "Ram",
```

```
    "Sita",
```

```
    "John",
```

```
"Priya"  
};
```

But arrays have limitations.

For example:

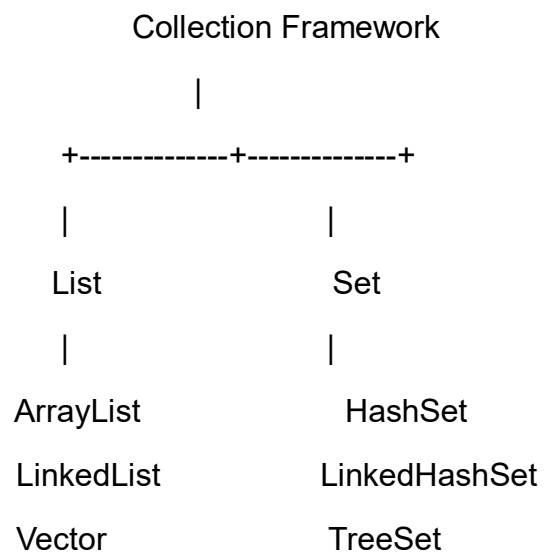
- fixed size
- inserting/deleting can be inconvenient
- searching may require loops
- duplicate handling is manual
- sorting/searching behavior is limited

Java provides the **Collections Framework** to make handling groups of objects easier.

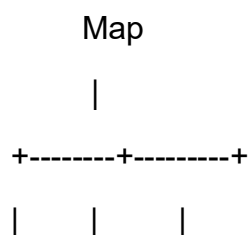
2. Collections Framework

Think of Collections Framework like a **toolbox**.

Different tools solve different problems.



And separately:



HashMap LinkedHashMap TreeMap

Important:

Map is part of the Collections Framework, but technically Map does **not** extend the Collection interface.

This distinction is very important in interviews.

3. First understand Set and Map

There are two completely different ideas.

Set

A Set stores **unique values**.

Example:

10

20

30

If you try:

10

20

30

10

the second 10 is not stored.

So:

Set = collection of unique values

Map

A Map stores data as:

KEY → VALUE

Example:

101 → Ravi

102 → Sita

103 → John

Here:

101 = key

Ravi = value

So:

Map = key-value pairs

PART 1 — SET

4. What is Set?

Set is an interface in Java.

```
Set<Integer> numbers;
```

A Set generally has these characteristics:

- does not allow duplicate elements
- useful for uniqueness
- exact ordering depends on implementation
- can store objects
- provides collection operations such as add/remove/contains

Example:

```
Set<Integer> numbers = new HashSet<>();
```

```
numbers.add(10);
```

```
numbers.add(20);
```

```
numbers.add(30);
```

```
numbers.add(10);
```

```
System.out.println(numbers);
```

Possible output:

```
[20, 10, 30]
```

Notice:

10

appears only once.

5. Why does Set not allow duplicates?

Suppose:

```
Set<Integer> numbers = new HashSet<>();
```

```
numbers.add(10);
```

```
numbers.add(10);
```

When the second 10 comes:

Already present?

|

YES

|

Don't add

So:

```
numbers.size()
```

will be:

1

not:

2

6. Set is an Interface

You normally don't write:

```
Set<Integer> numbers = new Set<>();
```

Why?

Because Set is an interface.

Instead:

```
Set<Integer> numbers = new HashSet<>();
```

Here:

Set<Integer>

↑

reference type

HashSet<>()

↑

actual object

This is called **programming to an interface**.

7. Three important Set implementations

You need to remember these three:

Set

|

+---- HashSet

|

+---- LinkedHashSet

|

+---- TreeSet

The biggest difference is **ordering**.

Set	Duplicate	Order
HashSet	✗	No guaranteed order
LinkedHashSet	✗	Insertion order
TreeSet	✗	Sorted order

Let's understand each deeply.

8. HashSet

HashSet is probably the most commonly used Set.

```
HashSet<Integer> numbers = new HashSet<>();
```

Example:

```
HashSet<Integer> numbers = new HashSet<>();
```

```
numbers.add(50);
```

```
numbers.add(10);
```

```
numbers.add(30);
```

```
numbers.add(10);
```

```
System.out.println(numbers);
```

Output could be:

```
[50, 10, 30]
```

The important point is:

Do not depend on this order.

HashSet does not guarantee insertion order.

9. Why is it called HashSet?

Because internally it uses **hashing**.

You don't need to manually calculate the hash.

When you do:

```
numbers.add(50);
```

Java internally uses the object's hash information to determine where it should be stored.

Conceptually:

```
50
```

```
|
```

```
hash
```

```
|
```

```
bucket
```

|

stored

This makes operations such as:

add()

remove()

contains()

very efficient on average.

Typical average complexity:

add() → $O(1)$

remove() → $O(1)$

contains() → $O(1)$

Worst-case behavior can differ depending on collisions and implementation details, so remember **average $O(1)$** for interview purposes.

10. What is hashing?

Let's understand with a simple example.

Suppose we have:

int number = 25;

Java can calculate hash-related information for an object.

Conceptually:

Object

↓

hash value

↓

bucket/location

Think of buckets like lockers.

Bucket 0

Bucket 1

Bucket 2

Bucket 3

Bucket 4

...

Hashing helps Java decide where an object belongs.

You don't need to know the exact bucket calculation for basic HashSet usage.

11. What happens when duplicate is inserted into HashSet?

Suppose:

```
HashSet<Integer> set = new HashSet<>();
```

```
set.add(10);
```

```
set.add(20);
```

```
set.add(10);
```

Step-by-step:

First:

```
set.add(10);
```

10 isn't present.

Set:

10

Second:

```
set.add(20);
```

20 isn't present.

Set:

10

20

Third:

```
set.add(10);
```

Java checks whether equivalent 10 already exists.

It does.

Therefore:

Don't add duplicate

12. What does add() return?

This is useful.

```
boolean result = set.add(10);
```

If the element was successfully added:

true

If it was already present:

false

Example:

```
HashSet<Integer> set = new HashSet<>();
```

```
System.out.println(set.add(10));
```

```
System.out.println(set.add(10));
```

Output:

true

false

Because:

First 10 → new → added

Second 10 → duplicate → not added

13. Important HashSet methods

add()

```
set.add(10);
```

Adds an element.

remove()

```
set.remove(10);
```

Removes the element.

contains()

```
set.contains(10);
```

Checks whether element exists.

Returns:

true / false

size()

```
set.size();
```

Number of elements.

isEmpty()

```
set.isEmpty();
```

Checks whether Set contains nothing.

clear()

```
set.clear();
```

Removes everything.

14. Real-world HashSet example

Imagine a website where users enter their skills:

Java

Python

Java

HTML

Python

You want unique skills.

```
HashSet<String> skills = new HashSet<>();
```

```
skills.add("Java");  
skills.add("Python");  
skills.add("Java");  
skills.add("HTML");  
skills.add("Python");
```

```
System.out.println(skills);
```

Result contains:

Java

Python

HTML

No duplicates.

15. LinkedHashSet

Now imagine:

Java

Python

HTML

CSS

You want:

- no duplicates
- original insertion order

Then use:

LinkedHashSet

Example:

```
LinkedHashSet<String> set = new LinkedHashSet<>();
```

```
set.add("Java");
```

```
set.add("Python");
```

```
set.add("HTML");
```

```
set.add("CSS");
```

```
System.out.println(set);
```

Output:

[Java, Python, HTML, CSS]

The insertion order is maintained.

16. Why LinkedHashSet?

Suppose you add:

Apple

Banana

Mango

Orange

You want output in exactly the same order.

Use:

LinkedHashSet

It internally combines the idea of:

Hashing

+

Linked structure

The hashing helps with efficient lookup, while linked information maintains insertion order.

17. HashSet vs LinkedHashSet

HashSet

```
HashSet<String> set = new HashSet<>();
```

You cannot rely on insertion order.

LinkedHashSet

```
LinkedHashSet<String> set = new LinkedHashSet<>();
```

Insertion order is maintained.

Example:

Input:

B

A

D

C

HashSet:

Order not guaranteed

LinkedHashSet:

B

A

D

C

18. TreeSet

Now suppose you want:

10

20

30

40

50

automatically sorted.

Use:

TreeSet

Example:

```
TreeSet<Integer> numbers = new TreeSet<>();
```

```
numbers.add(40);  
numbers.add(10);  
numbers.add(30);  
numbers.add(20);
```

```
System.out.println(numbers);
```

Output:

```
[10, 20, 30, 40]
```

TreeSet automatically maintains elements in sorted order.

19. Why TreeSet?

Think of a school.

Students have marks:

85

60

95

72

If you want them automatically arranged:

60

72

85

95

TreeSet is useful.

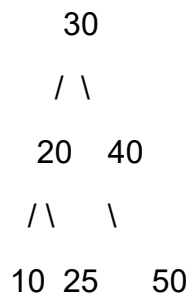
20. How TreeSet works internally

TreeSet is based on a tree structure.

More specifically, Java's TreeSet is backed by a TreeMap, which uses a **Red-Black Tree**.

Don't worry about implementing the tree yourself right now.

Conceptually:



The tree maintains ordering.

Therefore operations are typically:

add() → $O(\log n)$

remove() → $O(\log n)$

contains() → $O(\log n)$

21. HashSet vs LinkedHashSet vs TreeSet

This table is extremely important for interviews.

Feature	HashSet	LinkedHashSet	TreeSet
Duplicate	✗	✗	✗
Insertion order	✗	✓	✗
Sorted order	✗	✗	✓
Internal idea	Hash table	Hash + linked structure	Tree
Average add	$O(1)$	$O(1)$	$O(\log n)$
Average search	$O(1)$	$O(1)$	$O(\log n)$
Null	One null generally allowed	One null generally allowed	Generally no null for natural ordering
Main use	Fast uniqueness	Unique + insertion order	Unique + sorted order

Easy memory trick:

HashSet

= Unique + Fast

LinkedHashSet

= Unique + Insertion Order

TreeSet

= Unique + Sorted

22. When should you use which Set?

Requirement:

"I only care about uniqueness."

Use:

HashSet

Requirement:

"I need uniqueness and insertion order."

Use:

LinkedHashSet

Requirement:

"I need uniqueness and sorted data."

Use:

TreeSet

PART 2 — MAP

23. What is Map?

Now we move to one of the most important Java topics:

Map

A Map stores data as:

KEY → VALUE

Example:

101 → Ravi

102 → Sita

103 → John

Here:

101 = key

Ravi = value

24. Real-world example of Map

Think about a phone contact list.

Name → Phone Number

Example:

Ravi → 9876543210

Sita → 9123456780

John → 9988776655

This is naturally represented by a Map.

```
Map<String, String> contacts = new HashMap<>();
```

Then:

```
contacts.put("Ravi", "9876543210");
```

```
contacts.put("Sita", "9123456780");
```

25. Map structure

Think:

KEY VALUE

"Ravi" → "9876543210"

"Sita" → "9123456780"

"John" → "9988776655"

Each entry is called a:

key-value pair

or

Map entry

26. Why do we need Map?

Suppose:

Student ID → Student Name

You could use two arrays:

```
int[] ids = {101, 102, 103};
```

```
String[] names = {"Ravi", "Sita", "John"};
```

But now you must keep both arrays synchronized.

Map makes the relationship explicit:

```
Map<Integer, String> students = new HashMap<>();
```

```
students.put(101, "Ravi");
```

```
students.put(102, "Sita");
```

```
students.put(103, "John");
```

Now:

```
students.get(101);
```

gives:

Ravi

27. Map does not allow duplicate keys

This is VERY important.

```
Map<Integer, String> students = new HashMap<>();
```

```
students.put(101, "Ravi");
```

```
students.put(101, "Sita");
```

What happens?

The second value replaces the first.

Final:

101 → Sita

Why?

Because:

A Map cannot have duplicate keys.

But it **can have duplicate values**.

Example:

101 → Ravi

102 → Ravi

This is valid.

28. Key vs Value

Remember:

KEY

↓

used to identify/retrieve value

VALUE

↓

actual data

Example:

```
students.put(101, "Ravi");
```

Here:

101 = key

Ravi = value

When you do:

```
students.get(101);
```

Java searches using:

101

and returns:

Ravi

29. Important Map methods

put()

Adds key-value pair.

```
map.put(101, "Ravi");
```

get()

Gets value using key.

```
map.get(101);
```

Returns:

Ravi

remove()

```
map.remove(101);
```

Removes key-value pair.

containsKey()

```
map.containsKey(101);
```

Checks whether key exists.

containsValue()

```
map.containsValue("Ravi");
```

Checks whether value exists.

size()

```
map.size();
```

Number of key-value pairs.

isEmpty()

```
map.isEmpty();
```

Checks whether Map is empty.

clear()

```
map.clear();
```

Removes everything.

30. HashMap

HashMap is the most commonly used Map implementation.

```
HashMap<Integer, String> students = new HashMap<>();
```

Example:

```
students.put(101, "Ravi");
```

```
students.put(102, "Sita");
```

```
students.put(103, "John");
```

```
System.out.println(students);
```

Important:

HashMap does not guarantee insertion order.

31. HashMap internal idea

HashMap uses **hashing**.

When you write:

```
map.put(101, "Ravi");
```

conceptually:

Key

↓
hash
↓
bucket
↓
key-value entry

Then:

```
map.get(101);
```

Java uses the key to find the appropriate location efficiently.

Average:

```
put()    → O(1)
```

```
get()    → O(1)
```

```
remove() → O(1)
```

```
containsKey → O(1)
```

Again, these are average/expected complexities.

32. HashMap with duplicate key

Example:

```
HashMap<Integer, String> map = new HashMap<>();
```

```
map.put(101, "Ravi");
```

```
map.put(101, "Sita");
```

First:

```
101 → Ravi
```

Second:

```
101 → Sita
```

The second put() updates the existing key.

Final:

```
101 → Sita
```

33. What does put() return?

Interesting interview question.

```
map.put(101, "Ravi");
```

put() returns the **previous value associated with that key**, or null if there wasn't one.

Example:

```
HashMap<Integer, String> map = new HashMap<>();
```

```
System.out.println(map.put(101, "Ravi"));
```

```
System.out.println(map.put(101, "Sita"));
```

Output:

null

Ravi

Why?

First:

101 didn't exist

so previous value = null.

Second:

101 already had Ravi

so previous value = Ravi.

Then it changes to:

101 → Sita

34. LinkedHashMap

What if you want:

Map

+

Insertion order

Use:

LinkedHashMap

Example:

```
LinkedHashMap<Integer, String> map = new LinkedHashMap<>();
```

```
map.put(101, "Ravi");
```

```
map.put(102, "Sita");
```

```
map.put(103, "John");
```

```
System.out.println(map);
```

The insertion order is maintained.

101 → Ravi

102 → Sita

103 → John

35. HashMap vs LinkedHashMap

HashMap

Fast

No guaranteed insertion order

LinkedHashMap

Fast

Maintains insertion order

Memory trick:

HashMap

= Key → Value + Fast

LinkedHashMap

= Key → Value + Fast + Insertion Order

36. TreeMap

Now:

Map

+

Sorted keys

Use:

TreeMap

Example:

```
TreeMap<Integer, String> map = new TreeMap<>();
```

```
map.put(103, "John");
```

```
map.put(101, "Ravi");
```

```
map.put(102, "Sita");
```

```
System.out.println(map);
```

Output:

```
{  
  101=Ravi,  
  102=Sita,  
  103=John  
}
```

Notice:

The **keys** are sorted.

37. Important: TreeMap sorts keys, not values

Suppose:

```
TreeMap<Integer, String> students = new TreeMap<>();
```

```
students.put(103, "John");
```

```
students.put(101, "Ravi");
```

```
students.put(102, "Sita");
```

Output:

101 → Ravi

102 → Sita

103 → John

Why?

Because keys are:

103

101

102

TreeMap sorts them:

101

102

103

It does NOT automatically sort:

John

Ravi

Sita

38. TreeMap internal idea

TreeMap uses a balanced tree structure, specifically a **Red-Black Tree**.

Typical complexity:

put() → $O(\log n)$

get() → $O(\log n)$

remove() → $O(\log n)$

containsKey → $O(\log n)$

It maintains sorted key order.

39. HashMap vs LinkedHashMap vs TreeMap

Very important interview table:

Feature	HashMap	LinkedHashMap	TreeMap
Key-value	✓	✓	✓
Duplicate keys	✗	✗	✗
Duplicate values	✓	✓	✓
Insertion order	✗	✓	✗
Sorted keys	✗	✗	✓
Average basic operations	$O(1)$	$O(1)$	$O(\log n)$
Main use	Fast lookup Lookup + insertion order Sorted keys		

Memory trick:

HashMap

→ Fast

LinkedHashMap

→ Fast + Insertion Order

TreeMap

→ Sorted Keys

40. Set vs Map

This is one of the most common interview questions.

Set

Stores:

VALUE

Example:

10

20

30

Map

Stores:

KEY → VALUE

Example:

101 → Ravi

102 → Sita

Simple memory:

Set = "I only need unique things."

Map = "I need to connect one thing with another thing."

41. Set vs Map Example

Suppose you have:

Java

Python

Java

C++

Python

You want unique languages.

Use:

Set<String>

Result:

Java

Python

C++

But suppose you want:

Java → 2

Python → 2

C++ → 1

Now you need:

Map<String, Integer>

This leads directly to your first practice problem.

PART 3 — FREQUENCY COUNTER

42. What is frequency?

Frequency means:

How many times something appears.

Example:

Input:

apple banana apple orange banana apple

Frequency:

apple → 3

banana → 2

orange → 1

This is a very common DSA/interview problem.

43. Why Map is perfect for frequency?

Because we need:

item → count

That is exactly a Map.

apple → 3

banana → 2

orange → 1

So:

Map<String, Integer>

is perfect.

44. Frequency counter step-by-step

Input:

apple

banana

apple

orange

banana

apple

Start:

Map = {}

Read apple:

apple → 1

Read banana:

apple → 1

banana → 1

Read apple again:

apple → 2

banana → 1

Read orange:

apple → 2

banana → 1

orange → 1

Read banana:

apple → 2

banana → 2

orange → 1

Read apple:

apple → 3

banana → 2

orange → 1

Final:

apple → 3

banana → 2

orange → 1

45. Basic frequency counter code

```
import java.util.HashMap;
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        String[] fruits = {
```

```
            "apple",
```

```
            "banana",
```

```
            "apple",
```

```
            "orange",
```

```
            "banana",
```

```
            "apple"
```

```
        };
```

```
        HashMap<String, Integer> frequency = new HashMap<>();
```

```
        for (String fruit : fruits) {
```

```
            if (frequency.containsKey(fruit)) {
```

```
                frequency.put(fruit, frequency.get(fruit) + 1);
```

```
            } else {
```

```
        frequency.put(fruit, 1);
    }
}

    System.out.println(frequency);
}
}
```

Output:

{orange=1, banana=2, apple=3}

Remember: HashMap doesn't guarantee display order.

46. Understand this line deeply

frequency.get(fruit)

Suppose:

frequency:

apple → 2

and:

fruit = "apple";

Then:

frequency.get("apple")

returns:

2

Therefore:

frequency.get(fruit) + 1

becomes:

2 + 1

which is:

3

Then:

```
frequency.put(fruit, 3);
```

updates:

apple → 3

47. Frequency counter with `getOrDefault()`

Java gives us a very useful Map method:

`getOrDefault()`

Syntax:

```
map.getOrDefault(key, defaultValue)
```

Example:

```
frequency.getOrDefault("apple", 0);
```

If apple exists:

return its value

If apple doesn't exist:

return 0

So we can write:

```
for (String fruit : fruits) {  
    frequency.put(  
        fruit,  
        frequency.getOrDefault(fruit, 0) + 1  
    );  
}
```

This is much cleaner.

48. Understand the frequency line

```
frequency.put(  
    fruit,  
    frequency.getOrDefault(fruit, 0) + 1  
);
```

Suppose:

fruit = apple

and apple doesn't exist.

Then:

`frequency.getOrDefault("apple", 0)`

returns:

0

Then:

$0 + 1 = 1$

So:

apple $\rightarrow$ 1

Next apple:

`getOrDefault("apple", 0)`

returns:

1

Then:

$1 + 1 = 2$

So:

apple $\rightarrow$ 2

Very important pattern:

```
map.put(  
    key,  
    map.getOrDefault(key, 0) + 1  
);
```

You will see this pattern frequently in DSA.

PART 4 — DUPLICATE REMOVAL

49. What is duplicate removal?

Input:

10 20 10 30 20 40

Output:

10 20 30 40

We want each element only once.

This is exactly what a Set does.

50. Duplicate removal using HashSet

```
import java.util.HashSet;
```

```
public class Main {  
    public static void main(String[] args) {  
  
        int[] numbers = {10, 20, 10, 30, 20, 40};  
  
        HashSet<Integer> unique = new HashSet<>();  
  
        for (int number : numbers) {  
            unique.add(number);  
        }  
  
        System.out.println(unique);  
    }  
}
```

Output contains:

10

20

30

40

Order is not guaranteed.

51. Duplicate removal while preserving insertion order

Suppose:

Input:

10 20 10 30 20 40

You want:

10 20 30 40

in the same order of first appearance.

Use:

`LinkedHashSet<Integer>`

Example:

```
LinkedHashSet<Integer> unique = new LinkedHashSet<>();
```

```
for (int number : numbers) {  
    unique.add(number);  
}
```

Result:

10

20

30

40

52. Duplicate removal + sorting

Suppose:

Input:

40 10 30 10 20 40

You want:

10 20 30 40

Use:

TreeSet<Integer>

Example:

```
TreeSet<Integer> unique = new TreeSet<>();
```

```
for (int number : numbers) {  
    unique.add(number);  
}
```

Now:

```
unique = [10, 20, 30, 40]
```

So:

HashSet

→ unique

LinkedHashSet

→ unique + original insertion order

TreeSet

→ unique + sorted order

53. Very important: Set and Objects

You can store objects in Sets.

Example:

```
class Student {
```

```
    int id;
```

```
    String name;
```

```
    Student(int id, String name) {
```

```
        this.id = id;
```

```
        this.name = name;
```

```
}  
}
```

Then:

```
HashSet<Student> students = new HashSet<>();
```

But there is an important concept:

equals() and hashCode()

For custom objects, if you want HashSet/HashMap to correctly recognize logically equal objects, you generally need to implement equals() and hashCode() consistently.

Example concept:

Student 1

id = 101

name = Ravi

Student 2

id = 101

name = Ravi

They are two different objects in memory.

Java needs to know whether you consider them logically equal.

That's where:

equals()

hashCode()

become important.

54. Why are equals() and hashCode() connected to HashSet?

HashSet uses hashing to organize objects.

Conceptually:

hashCode()

↓

find possible bucket

↓

equals()

↓

check actual equality

The important contract is:

If two objects are equal according to equals(), they must have the same hashCode().

But two objects having the same hash code does **not** necessarily mean they are equal.

This is called a **hash collision**.

55. What is a hash collision?

Imagine two different objects produce the same hash value.

Object A → hash 100

Object B → hash 100

That's a collision.

Java handles collisions internally.

Conceptually:

hash

↓

same bucket

↓

compare objects

↓

equals()

You don't normally handle the bucket mechanics yourself.

56. Why Map also uses hashCode() and equals()

For:

HashMap<Student, String>

the key must be correctly handled.

HashMap uses the key to locate the entry.

Conceptually:

key

↓

hashCode()

↓

bucket

↓

equals()

↓

matching key

↓

value

Therefore:

HashSet

HashMap

are closely connected to:

hashCode()

equals()

57. Map's entrySet()

Suppose:

```
Map<Integer, String> students = new HashMap<>();
```

```
students.put(101, "Ravi");
```

```
students.put(102, "Sita");
```

If you want both key and value:

```
for (Map.Entry<Integer, String> entry : students.entrySet()) {
```

```
        System.out.println(
            entry.getKey() + " " + entry.getValue()
        );
    }
}
```

Output:

101 Ravi

102 Sita

Why entrySet()?

Because each Map entry contains:

key + value

58. keySet()

If you only need keys:

```
for (Integer key : students.keySet()) {
    System.out.println(key);
}
```

Output:

101

102

59. values()

If you only need values:

```
for (String value : students.values()) {
    System.out.println(value);
}
```

Output:

Ravi

Sita

So remember:

keySet()

↓

keys

values()

↓

values

entrySet()

↓

key + value

60. Null values and null keys

This is another interview topic.

HashMap

Generally allows:

one null key

multiple null values

Example:

```
HashMap<String, Integer> map = new HashMap<>();
```

```
map.put(null, 10);
```

```
map.put("A", null);
```

```
map.put("B", null);
```

This is allowed.

LinkedHashMap

Similarly supports null keys and values.

TreeMap

With natural ordering, a null key is generally not permitted because the map needs to compare keys for ordering.

For example:

```
TreeMap<Integer, String> map = new TreeMap<>();
```

A null key cannot be naturally compared with integers.

61. Why do we use generics?

You've seen:

```
HashSet<Integer>
```

```
HashMap<Integer, String>
```

What is <Integer>?

That's **generic type information**.

Example:

```
HashSet<Integer> numbers = new HashSet<>();
```

means:

This Set is intended to store Integer objects.

And:

```
HashMap<Integer, String> students
```

means:

Key → Integer

Value → String

Therefore:

```
students.put(101, "Ravi");
```

is valid.

But:

```
students.put("101", "Ravi");
```

is not valid because the key should be Integer.

62. Why Integer instead of int?

Collections store objects, not primitive types.

So:

`HashSet<Integer>`

not:

`HashSet<int>`

Java automatically converts:

`int`

to:

`Integer`

This is called:

Autoboxing

Example:

`int x = 10;`

`Integer y = x;`

Java automatically converts primitive `int` to `Integer`.

63. Unboxing

Reverse conversion:

`Integer x = 10;`

`int y = x;`

Java converts:

`Integer` $\rightarrow$ `int`

This is called:

Unboxing

Remember:

`int` $\rightarrow$ `Integer`

= Autoboxing

Integer → int

= Unboxing

This is why you can comfortably write:

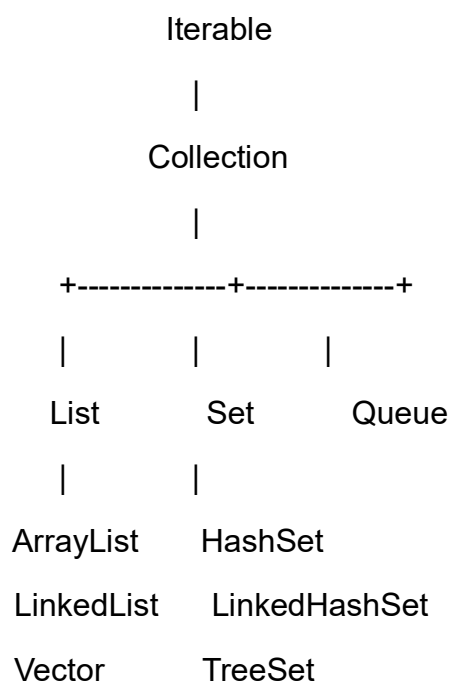
```
HashSet<Integer> set = new HashSet<>();
```

```
set.add(10);
```

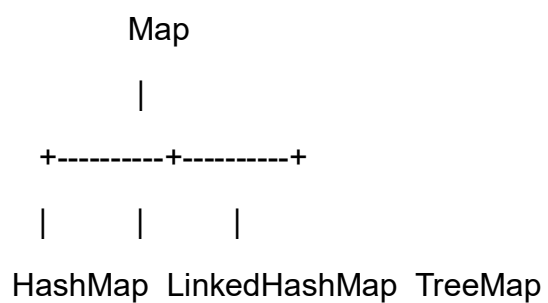
even though 10 is technically an int literal.

64. Complete Collection hierarchy you should remember

At your level, visualize it like this:



And separately:



Again:

Map is not a child of Collection.

65. Complete comparison

Set family

Collection	Duplicate	Order	Sorted	Typical basic lookup
HashSet	✗	No guarantee	✗	$O(1)$ average
LinkedHashSet	✗	Insertion	✗	$O(1)$ average
TreeSet	✗	Sorted	✓	$O(\log n)$

Map family

Collection	Duplicate Keys	Order	Sorted Keys	Typical basic lookup
HashMap	✗	No guarantee	✗	$O(1)$ average
LinkedHashMap	✗	Insertion	✗	$O(1)$ average
TreeMap	✗	Sorted by key	✓	$O(\log n)$

66. The most important selection rule

When you're solving a problem, ask:

Question 1

Do I need key $\rightarrow$ value?

If YES:

Map

If NO:

Set

Question 2

If Set:

Do I need sorted order?

YES:

TreeSet

NO:

Do I need insertion order?

YES:

LinkedHashSet

NO:

HashSet

Question 3

If Map:

Do I need sorted keys?

YES:

TreeMap

NO:

Do I need insertion order?

YES:

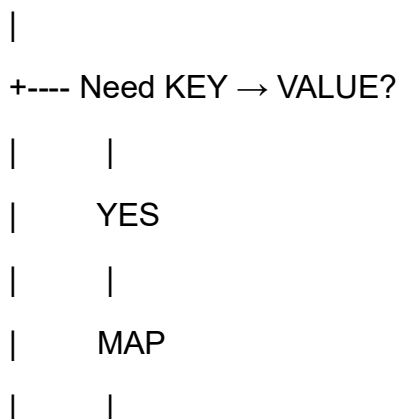
LinkedHashMap

NO:

HashMap

67. Easy decision tree

Need collection?



```

|  +-----+-----+
|  |         |
| sorted keys? insertion order?
|  |         |
|  TreeMap  LinkedHashMap
|  |
| otherwise
|  |
|  HashMap
|
NO
|
SET
|
+---+-----+
|         |
sorted?   insertion order?
|         |
TreeSet   LinkedHashSet
|
otherwise
|
HashSet

```

68. Common interview questions

Q1. What is Set?

Set is an interface that represents a collection of unique elements and does not allow duplicate elements.

Q2. Does Set maintain insertion order?

Not every Set.

HashSet → no guaranteed insertion order

LinkedHashSet → yes

TreeSet → sorted order

Q3. Difference between HashSet and TreeSet?

HashSet

→ hashing

→ no guaranteed order

→ average $O(1)$ basic operations

TreeSet

→ tree-based

→ sorted order

→ $O(\log n)$ basic operations

Q4. Difference between HashSet and LinkedHashSet?

HashSet

→ no guaranteed insertion order

LinkedHashSet

→ maintains insertion order

Q5. What is Map?

Map stores data as key-value pairs where keys are unique.

Q6. Can Map have duplicate keys?

No.

```
map.put(1, "A");
```

```
map.put(1, "B");
```

The second value replaces the first.

Q7. Can Map have duplicate values?

Yes.

1 → Ravi

2 → Ravi

is valid.

Q8. Difference between HashMap and LinkedHashMap?

HashMap

→ no guaranteed insertion order

LinkedHashMap

→ maintains insertion order

Q9. Difference between HashMap and TreeMap?

HashMap

→ hashing

→ no sorted keys

→ average $O(1)$

TreeMap

→ tree

→ sorted keys

→ $O(\log n)$

Q10. Why HashMap is fast?

Because it uses hashing to locate keys efficiently.

Q11. Why TreeMap is slower than HashMap for basic lookup?

Because TreeMap maintains sorted order using a tree structure, resulting in $O(\log n)$ operations rather than HashMap's average $O(1)$.

Q12. What is hashing?

Hashing is a technique that uses a hash value to help determine where an object/key should be stored or found.

Q13. What are equals() and hashCode()?

equals() determines logical equality.

hashCode() produces an integer hash value used by hash-based collections.

For objects considered equal by equals(), their hashCode() must be the same.

69. Common mistakes beginners make

Mistake 1

Thinking HashSet is sorted.

✗ Wrong.

HashSet<Integer>

does not guarantee sorted order.

For sorted elements:

TreeSet<Integer>

Mistake 2

Thinking LinkedHashSet sorts.

✗ Wrong.

LinkedHashSet maintains:

insertion order

not sorted order.

Mistake 3

Thinking TreeMap sorts values.

✗ Wrong.

TreeMap sorts:

keys

Mistake 4

Thinking Map allows duplicate keys.

✗ Wrong.

Keys are unique.

Mistake 5

Thinking Set stores key-value pairs.

✗ Wrong.

Set stores individual elements.

Mistake 6

Thinking Map extends Collection.

✗ Wrong.

Map is a separate interface hierarchy.

70. Real-world examples

HashSet

User's unique skills:

Java

Python

SQL

React

LinkedHashSet

Recently selected categories where order matters:

Java

Python

React

SQL

and duplicates should be removed.

TreeSet

Unique marks that need sorted order:

45

60

72

85

95

HashMap

Student lookup:

ID → Name

LinkedHashMap

Data where you want key-value lookup while preserving insertion order.

TreeMap

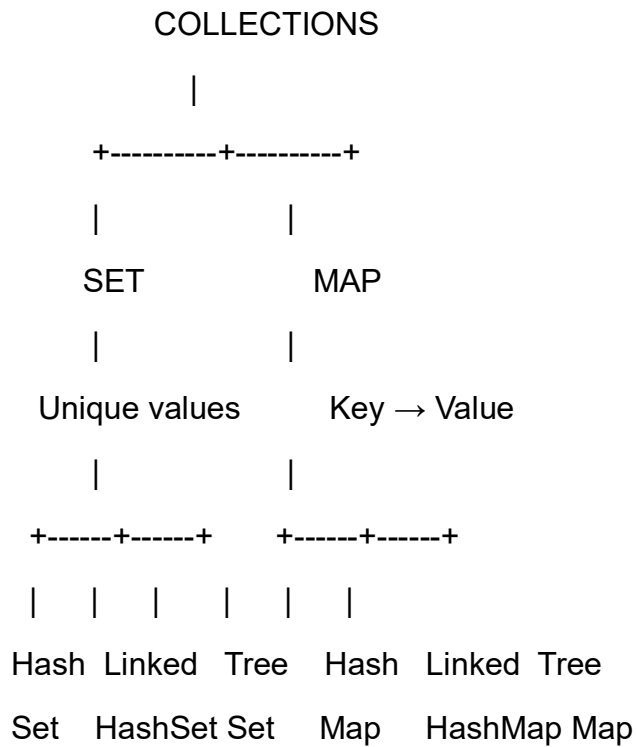
Sorted records:

Employee ID → Employee Name

where IDs should appear in ascending order.

71. Day 31–33 mental map

You should now think like this:



And remember:

Hash



Fast lookup

LinkedHash



Insertion order

Tree



Sorted order

That single pattern will help you remember most of this topic.

72. Practice Problems

Now don't just memorize. Solve these.

Practice 1 — Duplicate Removal

Input:

10 20 10 30 20 40 30

Output:

10 20 30 40

Hint

Think:

Set

Which Set?

If insertion order matters:

LinkedHashSet

Practice 2 — Frequency Counter

Input:

apple banana apple orange banana apple

Expected:

apple → 3

banana → 2

orange → 1

Hint

Think:

Map<String, Integer>

Pattern:

```
map.put(  
    item,  
    map.getOrDefault(item, 0) + 1  
);
```

Practice 3 — Character Frequency

Input:

"programming"

Find:

p → 1

r → 2

o → 1

g → 2

a → 1

m → 2

i → 1

n → 1

Hint:

Map<Character, Integer>

Practice 4 — First Repeated Number

Input:

10 20 30 20 40

Find the first number that appears again.

Hint:

Set

Process each number one by one:

Already present?

|

YES → repeated

NO → add

Practice 5 — Student Marks

Create:

Map<String, Integer>

Store:

Ravi → 85

Sita → 92

John → 78

Then retrieve John's marks.

Expected:

78

73. One final cheat sheet

Keep this in your mind:

SET

|

├── HashSet

| ├── Unique

| ├── No guaranteed order

| └── Average $O(1)$

|

├── LinkedHashSet

| ├── Unique

| ├── Insertion order

| └── Average $O(1)$

|

└── TreeSet

 ├── Unique

 ├── Sorted

 └── $O(\log n)$

MAP

|

|— HashMap

|

|— Key $\rightarrow$ Value

|

|— Unique keys

|

|— No guaranteed order

|

|— Average $O(1)$

|

|— LinkedHashMap

|

|— Key $\rightarrow$ Value

|

|— Unique keys

|

|— Insertion order

|

|— Average $O(1)$

|

|— TreeMap

|— Key $\rightarrow$ Value

|— Unique keys

|— Sorted keys

|— $O(\log n)$

The 6-line memory trick

HashSet $\rightarrow$ Unique + Fast

LinkedHashSet $\rightarrow$ Unique + Insertion Order

TreeSet $\rightarrow$ Unique + Sorted

HashMap $\rightarrow$ Key-Value + Fast

LinkedHashMap $\rightarrow$ Key-Value + Insertion Order

TreeMap $\rightarrow$ Key-Value + Sorted Keys